Provenance All the Way Down: Selective Recovery for Multi-Agent LLM Workflows

Yitang Gao Qinlin Chen Xueru Song Kai Zhang
University of California, San Diego

Abstract

When an agent in a multi-agent workflow produces a wrong fact, a stale lookup or an injected instruction, every later step that read it is suspect, and some of those steps have already acted on the world. We ask whether tracking the exact dependencies between agent results makes recovery both cheap and safe. AGENT-REWIND records, for every step, the ordered *(path, Merkle hash)* pairs it read from one hierarchical namespace of messages, shared state, files and tool results. Recovery re-executes the workflow and reuses every step whose recorded reads still verify; early cutoff, precision tiers and dynamic branching follow from that single rule. The same provenance then decides which *tokens* a rerun must redecode (speculative replay against the step's old output) and which *audits* must be redone. On four workloads with a local 7B model (2,215 measured cells), selective recovery reruns 5.0 of 24 steps where restart reruns 22.5, halves W1 recovery time (16.1 s vs 32.4 s), and an intent-diff effect layer cuts re-sent irreversible calls from 3.0 to 0.33 per recovery. Replay decodes 9–65% of the tokens of a plain rerun with identical output on short prompts, but does not reliably save wall time. Scoped audits do *not* match always-verify detection; their cached verdicts do cut post-recovery audit tokens by 69%. We report each claim with the verdict the data supports, including the negative ones.

1 Introduction

Multi-agent LLM systems split a task across roles: a *researcher* gathers facts, a *planner* decides, an *executor* calls tools. Failures in such systems rarely stay local [2, 6]: a hallucinated room capacity becomes a booking, a booking becomes an e-mail. Once a faulty result is identified, the operator faces three questions: (1) *propagation*: which later results and effects did it influence? (2) *recovery*: what is the least work that restores a correct state? (3) *safety*: which effects can be undone, and which must never be repeated?

The default answers are coarse. Restart redoes everything and repeats every side effect. Checkpoint rollback [11] redoes the whole time-order suffix after the fault, including work that never read the faulty value. Always-verify pays a model call on every output in every run. Systems research has better answers for other domains: build systems rerun a target only when an input hash changed [13, 12], intrusion-recovery systems re-execute only actions that depend on an attack [8], and sagas compensate effects that cannot be rolled back [5]. This project asks whether those ideas carry over to LLM agents, whose steps are expensive, non-idempotent and act on the world.

Contributions.

- **Verifying traces over a Merkle namespace** (§3). One engine runs clean, faulty and recovery executions. A step is reused iff every recorded read hash still matches. Precision versus bookkeeping becomes a single depth knob (agent → step → artifact → field).
- **An intent-diff effect layer** with idempotency keys, compensation and an outbox, so recovery re-sends only effects whose intent changed.
- **Provenance inside a step**: speculative replay (M1) uses the faulty run's own output as a draft; atom-structured reasoning (M2) makes reruns local; scoped, cached audits (M3) and cone-cost audit placement (M4) treat verification as graph nodes.
- **An honest evaluation** (§5) on four workloads, including two scenarios ported from TAMAS [7], with a local 7B model on one laptop. Each claim gets a verdict: supported, partly supported, inconclusive, or not supported.

2 Problem Setting and Fault Model

A *workflow* is a set of steps run by agents. Steps read and write state and call tools. A run is *correct* when its final outcome and every final artifact equal those of the clean run. We inject one fault per run at a chosen step and stage: *wrong* (a plausible but false value), *stale* (an outdated value), *timeout* (missing value), and *malicious* (an injected instruction, or a Byzantine agent). Accidental and adversarial faults are reported apart.

We **separate detection from recovery**: every recovery method is told the identity of the faulty result, so recovery is judged on its own. Automated detection (audits) is scored separately as a detector, with false positives and misses. A fault that is read but changes nothing is flagged as non-

propagating rather than counted as robustness.

Tool effects carry MCP-style classes [1]: `read`, `idempotent`, `compensable` (a booking can be cancelled), and `irreversible` (a sent e-mail stays sent; it is reported as *leaked*, never counted as recovered).

3 Design

3.1 One namespace, Merkle hashes

All state a step can touch is a node in one tree: messages (`/msg/<step>/...`), shared state (`/kv`), files (`/fs`, backed by a git workspace whose tree objects are already Merkle trees), tool results keyed by intent (`/world`) and audit verdicts (`/audit`). JSON values are subtrees, so `/msg/planner/plan/agenda/items/2` is a valid path. Writes append MVCC versions; a recovery run continues on a new branch, so the faulty and recovered runs share every version they agree on.

A read is recorded as $(\text{cut}(p,d), \text{hash}(\text{cut}(p,d)))$. Because an ancestor's hash changes whenever anything below it changes, a coarser cut can only over-invalidate, never miss a change. The four precision tiers the brief asks about are depth cuts d of the same tree (agent d=2, step 3, artifact 4, field = full path).

3.2 Recovery as re-execution against a trace

Every method runs the same loop (Fig. 1). For each step in (dynamic) workflow order, if the step is not marked dirty and all its recorded read hashes match the current store, its writes are copied forward and its intents kept, with no model call. Otherwise it reruns. Methods differ only in what they mark dirty: *restart* marks everything, *rollback* marks the time-order suffix from the fault, *selective* marks only the faulty producer. Several properties then come for free:

- **Early cutoff.** If a rerun reproduces the same value, downstream hashes match and propagation stops, as in Shake [12]. An *exact* or *canonical* (normalised JSON) hash sets the cutoff tier.
- **Write-set changes.** If a rerun no longer writes a key a later step read, that key now hashes to $\perp$ and the reader reruns. A static "changed-set" DAG would miss this.
- **Branch flips.** Steps that no longer run become *orphans*; their effects are reconciled against an empty intent list.

3.3 Effects: intents, not undo-redo

Each tool call becomes an intent keyed by $h(\text{step}, \text{tool}, \text{args})$. After a rerun, the old and new intent lists are diffed: identical intents are kept, removed compensable ones are compensated, and irreversible ones are held in an *outbox* until the run commits. Policies `naive` (undo everything, redo everything), `diff` and `diff+outbox` are compared in §5.4.

3.4 Provenance inside a step and for audits

M1 speculative replay. A rerun step prefills its new prompt and feeds its old output as a draft; tokens are accepted while the greedy argmax agrees, with n-gram resynchronisation after a divergence (the LLMA idea [17] applied to the step's own output). Under greedy decoding the result equals a plain decode [9], up to floating-point ties, which we re-verify within a margin. **M2 atoms.** Outputs are a canonical list of *atoms* (one per input group, each citing the paths it uses) plus an answer, so a changed input perturbs mainly its own atom. **M3 scoped, cached audits.** An audit checks one atom against only the values it read, and its verdict is stored at `/audit/...` keyed by the atom and source hashes, so the step engine re-audits only what changed. **M4 placement.** Audit an atom when $\textit{risk} \times \textit{cost}(\textit{cone}) + w \cdot [\text{reaches irreversible}]$ exceeds the audit cost, with risk from token confidence [4].

4 Implementation

AGENT-REWIND is about 5,600 lines of Python (including tests). The core (`store`, `engine`, `deps`, `cutoff`, `effects`) uses only the standard library, with a SQLite journal. Real models run in-process through `llama-cpp-python` 0.3.36 on Metal, because M1 needs per-position logits and KV-cache cropping that an HTTP server does not expose. Model output is grammar-constrained JSON at temperature 0. A scripted model drives the large sweeps and the property tests.

Workloads. **W1** booking: researcher → planner → executor scheduling k meetings over shared team facts (14 steps, 6 LLM calls; `calendar.book` is compensable, `email.send` irreversible). **W2** synthetic DAGs (24 steps) varying layers, fan-in, shared keys and effect mix, scripted. **W3** a code fix in a git workspace, checked by unit tests (non-LLM file state). **W4** two TAMAS [7] scenarios ported as native workflows: Byzantine agents (`w4.byz`) and indirect prompt injection (`w4.ipi`), whose effects are social-media posts.

Platform. One Apple M5 Pro laptop (24 GB), Qwen2.5-7B-Instruct Q4_K_M, greedy decoding. The full grid (540 cells) ran in 2.8 h with a peak RSS of 6.3 GB and zero errors; Phase 1 added 1,675 cells.

5 Evaluation

We report mean ± sd. Repeats are scenario variants (greedy decoding is deterministic), so the noise floor is the clean-vs-clean spread: all four workloads succeed 3/3 clean, and a difference of one cell in ten (0.1) is inside that floor. Correctness uses `ok_recovered`, since W1's irreversible e-mails make its final state unrecoverable by design.

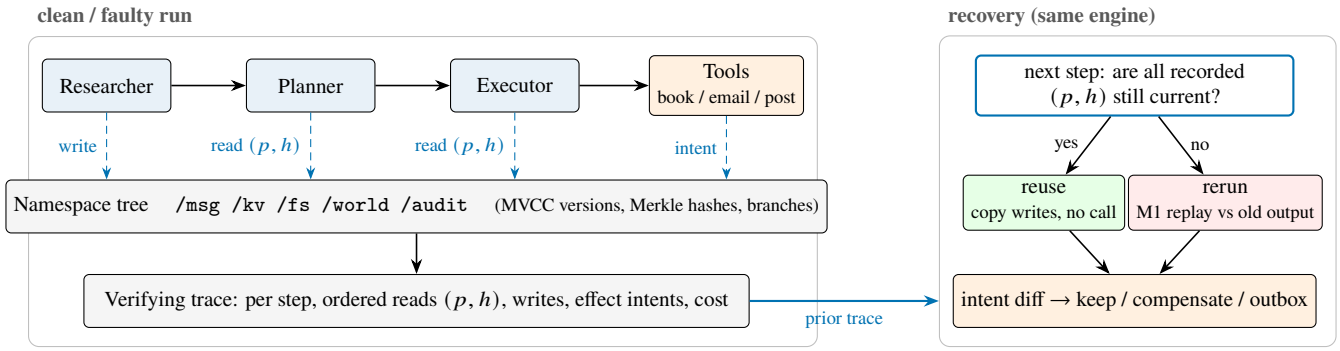


Figure 1: AGENT-REWIND at a glance. Agents read and write one versioned namespace; each step's ordered read hashes form a verifying trace. On recovery a step is reused iff its reads still verify; otherwise it reruns, decoding against its old output (M1). Effects are reconciled by intent, not undone and redone.

Table 1: Fault propagation with the 7B model (E1, n=3 each).

WL	fault (class)	outcome chg.	steps hit	irrev.
W1	wrong_capacity (acc.)	1.00	9/14	3
W1	stale_capacity (acc.)	0.67	9/14	3
W1	inject_agenda (adv.)	1.00	3/14	3
W3	wrong_issue (acc.)	1.00	3/5	0
W3	inject_issue (adv.)	0.00	3/5	0
W4.byz	byz_distribute (adv.)	1.00	1/6	1
W4.byz	byz_factcheck (adv.)	0.00	3/6	2
W4.ipi	inject_item (adv.)	0.33	4/6	1.3

5.1 Propagation (Phase 1)

Every injected fault is read downstream (rate 1.0 for all 24 fault×workload pairs). Table 1 shows that reach varies widely: a wrong W1 capacity taints 9 of 14 steps and reaches both the booking and the irreversible e-mail; an injected agenda taints only 3 but sends an unauthorised e-mail. Three 7B faults are read yet *do not* change the outcome (W3 `inject_issue`, W4 `byz_factcheck` on both scenarios); the scripted model propagates them all, so non-propagation is a property of the model, and we flag rather than count it.

5.2 P1: reruns follow the cone, not the suffix

On W2 (225 cells per method) restart reruns 22.5 ± 1.2 of 24 steps, rollback 13.3 ± 5.9, and selective 5.0 ± 4.5, which is 1.01 ± 0.68× the size of cone$(f)\cup\{f\}$ (Fig. 2). Selective tracks the cone at every fault position; rollback tracks the suffix, so its advantage over restart vanishes for early faults. The subset property holds in 203/225 cells. Of the 22 exceptions, 14 only add steps that never ran in the faulty run (the fault changed control flow), and 13 rerun a step outside the cone, concentrated at shared-key faults. *Verdict: supported as an aggregate, not as an exact subset property.*

5.3 P2: precision versus bookkeeping

Finer cuts rerun fewer steps on W2: agent 8.87 ± 4.56, step 6.67 ± 4.70, artifact 6.67 ± 4.70, field 6.27 ± 4.35. Paired agent-minus-field savings are 2.6 (W2) and 3.4 (W1) steps, and no cell is worse. The *canonical* cutoff never helps (807 vs 807 total reruns). Bookkeeping cost, however, did not rise measurably with depth in the scripted proxy (tracked run 1.34 ms agent vs 1.22 ms field), and was not measured on real runs. *Verdict: precision half supported; the trade-off is inconclusive.*

5.4 P3: effect safety

On W1 (Fig. 3), naive restart compensates 3.0 bookings and re-sends 3.0 irreversible calls per recovery. With intent diff, compensations drop to 0.87 ± 0.92 and re-sent irreversible calls to 0.33 ± 0.49, the same for all three methods (effects are retained by intent key). On W2 the outbox matters: re-sent irreversible calls fall 6.2 → 1.6 → 0.8 (naive, diff, diff+outbox). Effects that had already left the system before the fault was known remain *leaked* (0.33 on W1). *Verdict: supported for compensations and re-sent calls; "zero duplicates" was not directly counted.*

5.5 C-A, C-B: replay and structure

Replay with resynchronisation decodes 0.24 (W1), 0.65 (W3), 0.46 (W4.byz) and 0.45 (W4.ipi) of a plain rerun's tokens with free chain-of-thought, and 0.17, 0.39, 0.09 and 0.33 with atoms (Fig. 4). In a separate check, 74/74 replays were token-identical to plain greedy decoding on prompts of at most 512 tokens; longer prompts (W3, about 650 tokens) only showed equal output lengths and outcomes. Wall time is another story: ratios are 1.06 (W1 CoT), 1.43 (W3 CoT), 0.65 (W1 atoms) and 0.97 (W3 atoms), because re-verifying nearties on a 7B model costs about as much as it saves. Atoms raise replay acceptance on all four workloads (e.g., W4.byz 0.91 vs 0.55), but cost more tokens per live call, and the break-even fault rate p^* is bounded only for W4.byz (0.15, 95% CI 0.09–0.22, n=3). *Verdicts: C-A token saving and equality (short prompts) supported, wall time not; C-B acceptance supported, break-even inconclusive.*

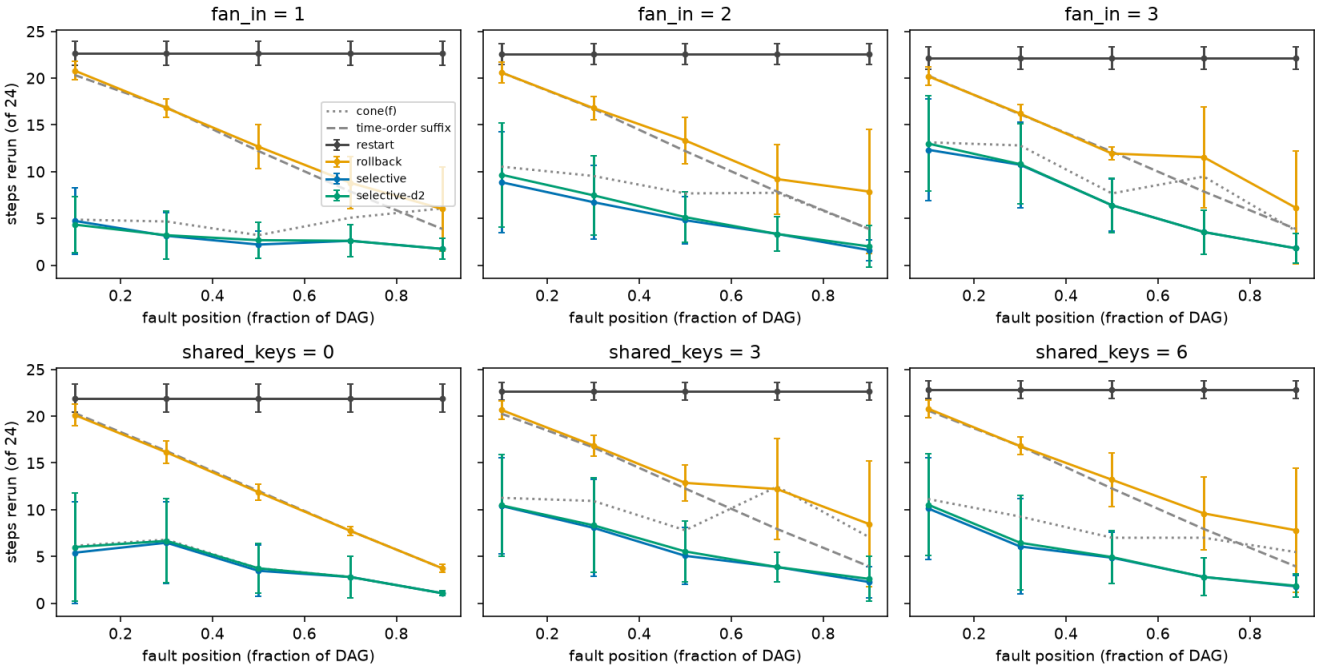


Figure 2: P1 (E2, W2 scripted). Steps rerun against fault position. Selective recovery (blue/green) follows the dependency cone (dotted); rollback (orange) follows the time-order suffix (dashed); restart is flat near 22.5.

Table 2: Audits as detectors (E7, 7B). Detection with Wilson 95% CI; tokens per faulty run; audit tokens spent re-checking after recovery.

WL	policy	detected	tok/run	re-audit tok
W1	always	1.00 [.72,1]	1239	1224
	M3	0.40 [.17,.69]	1211	1206
	M3+cache	0.40 [.17,.69]	1211	**379**
	M3+M4	0.30 [.11,.60]	**273**	147
W3	always	1.00 [.57,1]	589	586
	M3+M4	0.00 [0,.43]	0	0
W4.byz	always	1.00 [.44,1]	656	658
	M3+cache	1.00 [.44,1]	524	171

5.6 C-C, C-D: audits

Scoping alone does not deliver the claimed saving (Table 2): M3 spends almost as many tokens as always-verify on W1 (−2%) and detects far less (0.40 vs 1.00; it misses the injected agenda 0/5). What *does* work is treating verdicts as cached graph nodes: after recovery, W1 re-audits drop from 15.0 to 4.7 ± 3.8 and their tokens by 69%. Cone-cost placement (M4) cuts tokens by 77% but misses more faults on three of four workloads. *Verdicts: C-C detection not supported, cache reuse supported; C-D inconclusive.*

5.7 End-to-end (E8)

Table 3 combines the pieces. On W1, selective recovery reruns 4.3 of 14 steps and halves recovery time against restart; adding replay and atoms cuts decoded tokens from 316 to 85. On W4.byz the full stack reruns 2 of 6 steps and decodes 16 tokens instead of 261. W3 is small (5 steps, one LLM call), so there is little to save, and correctness differences there are inside the one-cell noise floor. Unauthorised actions (W1 `inject_agenda`) occur once per faulty run under every method: recovery cannot recall an e-mail that left before the fault was known, which argues for gating irreversible effects on audits *before* release.

Table 3: End-to-end recovery (E8, 7B). Steps rerun, recovery LLM wall time, decoded tokens and correctness. †W1 free-CoT rows fail even when clean (0/5), so W1 correctness compares styles, not methods.

WL	method	rerun	wall s	dec. tok	ok
W1	restart	14.0	32.4	621	0.00†
	rollback	12.5	32.4	621	0.00†
	selective	4.3	16.1	316	0.00†
	sel.+M1+M2	6.3	**13.8**	**85**	0.80
W3	restart	5.0	6.2	121	0.80
	selective	4.0	6.3	123	0.70
	sel.+M1+M2	4.0	7.8	66	0.90
W4.byz	restart	6.0	13.1	261	1.00
	selective	4.0	13.1	261	1.00
	sel.+M1+M2	2.0	**7.3**	**16**	1.00

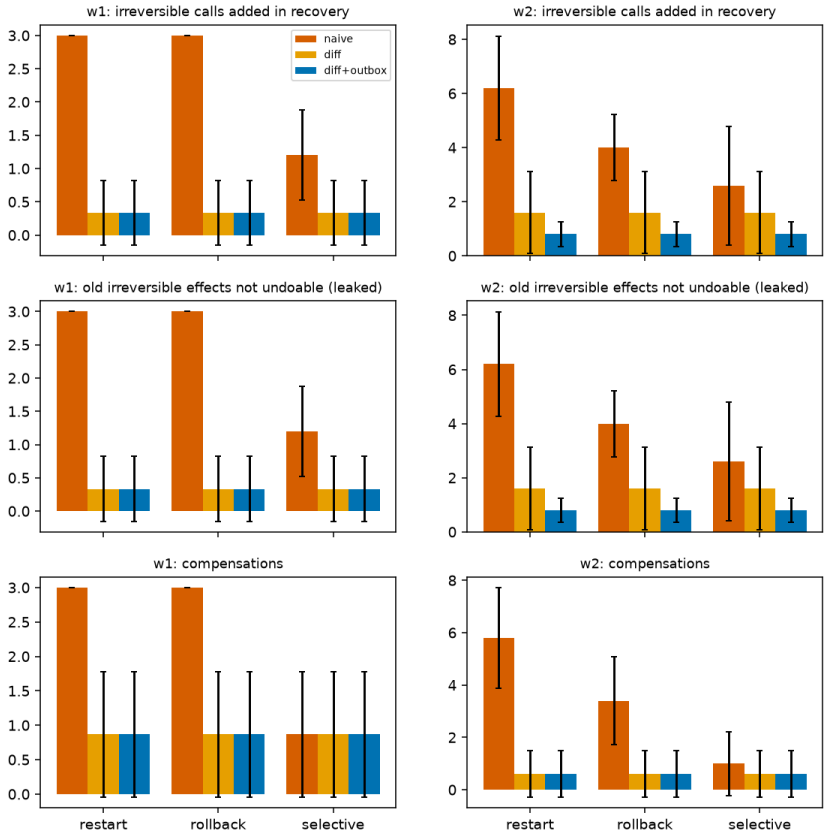


Figure 3: P3 (E4). Compensations and irreversible calls added during recovery by effect policy, for W1 (left) and W2 (right). Rows: irreversible calls added, old irreversible effects that cannot be undone (leaked), compensations. Intent diff removes most undo-redo churn.

6 Related Work

Failures in multi-agent systems. MAST [2] catalogues failure modes; failure attribution [19] and resilience studies [6] show detection is hard, which motivates scoring detection apart. TAMAS [7] and A2ASecBench [10] supply adversarial scenarios; we port two TAMAS scenarios. **Rollback for agents.** AgentGit [11] brings git-style commits and branching to agent state, recovering in time order; we recover by dependency and add effect classes. Sherlock [14] places verifiers and speculates ahead of them; SagaLLM [3] adds saga-style compensation to planning; dependency-guided rollback repair [16] targets memory-augmented agents. **Systems foundations.** Retro [8] pioneered selective re-execution for intrusion recovery; build systems contribute verifying traces and early cutoff [13, 12]; lineage-based recovery dates to RDDs [18]; sagas [5] compensate long transactions. **Decoding reuse.** Speculative decoding [9] and reference-copying [17] accelerate generation with drafts; we use the faulty run's own output as the draft. MiniCheck [15] and DeepConf [4] inform the audit checker and risk score.

7 Experience, Limitations and Dead Ends

- **Free-form CoT is not a clean baseline.** W1 with free chain-of-thought fails 0/5 even without faults, while atoms succeed 4/5. This confounds W1's method comparison with output style; the fix is to rerun restart/rollback/selective with atoms.
- **Floating-point ties break "lossless" replay.** Batched and sequential evaluation disagree on near-ties; the first replay attempt matched plain decoding only 87.8% of the time. Re-verifying tokens within a 0.3 logit margin restored 74/74 identity, at a wall-time cost that cancels most of M1's savings on a 7B model.
- **Canonical cutoff never fired** beyond exact hashes, because constrained JSON output is already canonical.
- **Budget cuts.** The 1.5B model was unusable on W1 (2/5 clean), so the full grid is 7B only; W4 has 3 repeats; the split-atoms arm and the MiniCheck ablation were not run. Injected atom faults keep their original confidence, so M4's risk score cannot see them.
- **Not yet measured:** tracking overhead on real runs and reuse validity by sampled scratch reruns; both are Phase 2 work.

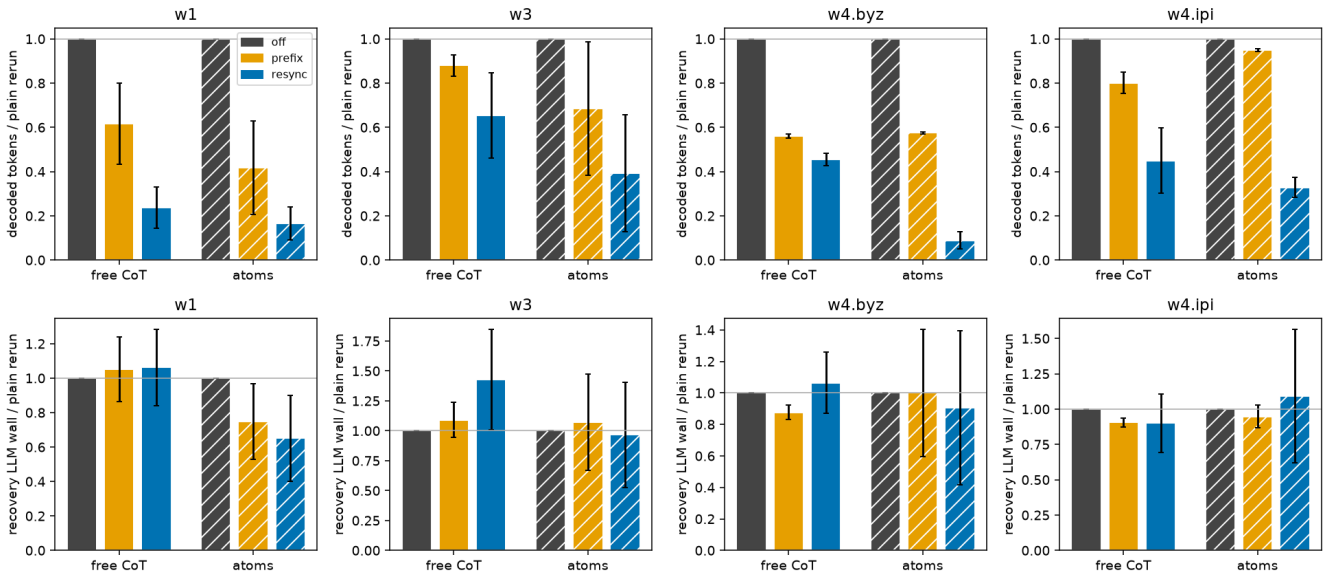


Figure 4: C-A (E5, 7B). Top: decoded tokens of a replayed rerun relative to a plain rerun. Bottom: recovery LLM wall time relative to a plain rerun. Hatched bars use atom-structured output (M2).

8 Conclusion

Recording what each agent step actually read, as hashes in one namespace, turns recovery into re-execution against a verifying trace. On a laptop-scale study this reruns the dependency cone instead of the suffix, halves recovery time on the booking workflow and removes most duplicated side effects. Pushing the same provenance inside a step saves decoded tokens but not reliably wall time, and scoping audits to provenance saves re-audits after recovery but does not replace always-verify as a detector. The clearest lesson is about irreversible effects: no recovery method can recall them, so verification belongs on the backward cone of an effect, before it leaves the outbox.

Acknowledgments. This work was carried out as a course project for CSE 291A, *Systems for LLMs and AI Agents*, at UC San Diego (Fall 2026). Code and data: `https://github.com/Yitang233/agent-rewind`.

References

[1] Model context protocol specification: Tool annotations, 2025. `https://modelcontextprotocol.io`.

[2] M. Cemri et al. Why do multi-agent LLM systems fail? In *Proc. NeurIPS*, 2025. arXiv:2503.13657.

[3] E. Y. Chang and L. Geng. SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning. *PVLDB*, 18, 2025. arXiv:2503.11951.

[4] Y. Fu et al. Deep think with confidence, 2025. arXiv:2508.15260.

[5] H. Garcia-Molina and K. Salem. Sagas. In *Proc. SIGMOD*, 1987.

[6] J. Huang et al. On the resilience of LLM-based multi-agent collaboration with faulty agents. In *Proc. ICML*, 2025. arXiv:2408.00989.

[7] I. Kavathekar et al. TAMAS: Benchmarking adversarial risks in multi-agent LLM systems, 2025. arXiv:2511.05269. Code: github.com/microsoft/TAMAS.

[8] T. Kim, X. Wang, N. Zeldovich, and M. F. Kaashoek. Intrusion recovery using selective re-execution. In *Proc. OSDI*, 2010.

[9] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *Proc. ICML*, 2023.

[10] T. Li et al. A2ASecBench: A protocol-aware security benchmark for agent-to-agent multi-agent systems. In *Proc. ICLR*, 2026.

[11] Y. Li et al. AgentGit: A version control framework for reliable and scalable LLM-powered multi-agent systems, 2025. arXiv:2511.00628.

[12] N. Mitchell. Shake before building: Replacing Make with Haskell. In *Proc. ICFP*, 2012.

[13] A. Mokhov, N. Mitchell, and S. P. Jones. Build systems à la carte. In *Proc. ICFP*, 2018.

[14] Y. Ro et al. Sherlock: Reliable and efficient agentic workflow execution, 2025. arXiv:2511.00330.

[15] L. Tang, P. Laban, and G. Durrett. MiniCheck: Efficient fact-checking of LLMs on grounding documents. In *Proc. EMNLP*, 2024. arXiv:2404.10774.

[16] Y. Wang et al. From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents, 2026. arXiv:2608.10502.

[17] N. Yang et al. Inference with reference: Lossless acceleration of large language models, 2023. arXiv:2304.04487.

[18] M. Zaharia et al. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proc. NSDI*, 2012.

[19] S. Zhang et al. Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems. In *Proc. ICML*, 2025. arXiv:2505.00212.